

```
#!/usr/bin/env python2.7
# === miscellaneous.py ===
import os
from multiprocessing import Process, Lock, Queue, Pipe, cpu_count

def lockfunc(olck, uilaps):
    olck.acquire()

    for ui in xrange(4*uilaps):
        print ' ' + str(ui) + ' lock acquired by pid: ' + str(os.getpid())

    olck.release()

def queuefunc(oque):
    oque.put(" message in Queue: LFY rockz!!!")

def pipefunc(oc):
    oc.send(" message in Pipe: FOSS rulz!!!")
    oc.close()

if '__main__' == __name__:

    uicores = cpu_count()
    olck = Lock()

    oque = Queue()
    op, oc = Pipe()

    for ui in xrange(1, 2*uicores):
        Process(target = lockfunc, args = (olck, ui,)).start()

    opq = Process(target = queuefunc, args = (oque,))
    opq.start()
    print
    print oque.get()
    print
    opq.join()

    opp = Process(target = pipefunc, args = (oc,))
    opp.start()
    print op.recv()
    print
    opp.join()
```

The *multiprocessing* package provides a lot of other functionality too; please refer to the tutorial link in the *References* section.

Parallelism via the *pprocess* module

The external *pprocess* Python module is somewhat inspired by the *thread* Python module, and provides a simple API and different styles for parallel programming. It is very similar to the *multiprocessing* package, so I will only touch

on important concepts; readers are encouraged to explore it further. The module provides lower-level constructs like threading- and forking-like parallelism, as well as higher-level constructs like *mpmap*, *channel*, *Exchange*, *MakeParallel*, *MakeReusable*, etc.

To use *pprocess*, download the latest version from the link in the *References* section. To install it in a terminal window, uncompress it with *tar xzvf pprocess-version.tar.gz*. Change to the extracted folder with *cd pprocess-version* (replace ‘version’ with the version number). With root privileges (use *sudo* or *su*) run *python setup.py install*. Verify if the installation has been successful with *python -m pprocess*.

The easiest way to parallelise code that uses the *map* function to execute a function over a list is to use *mpmap*, which does the same thing, but in parallel. There is an *examples* subdirectory in the *pprocess* sources that contains *map*-based *simple.py*, and the *mpmap* version, *simple_mpm.py*. On execution (see Figure 2), the *mpmap* version was roughly 10 times faster than the serial *map* version, clearly showing the performance boost from parallel execution on multiple cores.

The *channels* are objects returned after the creation of sub-processes, and they are used to communicate with the created processes. You can also create an *Exchange* object to know when to read from a channel. You can initialise the *Exchange* object with a list of channels, or *add* them to it later with its *add* method. You can use the *active* method to learn if it is actually monitoring any channels, and *ready* to see if any data is ready to be received.

This module also provides a *MakeParallel* class to create a wrapper around unmodified functions, which returns results via *channels*. Learn more about *pprocess* via the tutorial and references provided in the source’s *docs* subdirectory, and the code samples in the *examples* subdirectory.

Full-throttle multi-core/clustering with Parallel Python

Now we come to the most flexible, versatile and high-level tool in this article. The Parallel Python (PP) module is one of the easiest ways to write parallel applications in Python. A unique feature of this module is that you can also use it to parallelise your code over a computing cluster, through a very simple programming model: you start a PP execution server, submit jobs to it to be computed in parallel, and retrieve the computation results—that’s it!

To install PP, download the latest tarball from the link in the *References* section. Extract the archive and do a (*sudo* or *su*) *python setup.py install*, just like for the *pprocess* module. Verify the installation with *python -m pp*.

Now get your hands dirty; run the Python script shown below:

```
#!/usr/bin/env python
# === parallel.py ===
from pp import Server
```